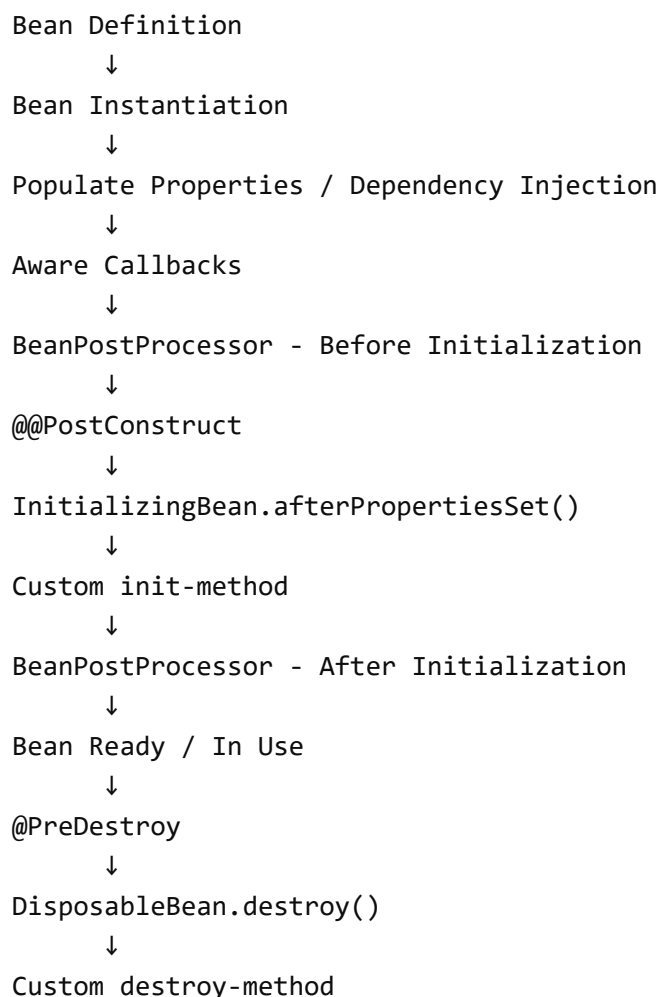


Bean Lifecycle Overview

In Spring, **Bean Lifecycle** is the sequence of steps Spring follows from **creating a bean definition** → **instantiating the bean** → **injecting dependencies** → **initializing it** → **using it** → **destroying it**.

For a 7–8 year experienced developer, the important lifecycle is:



```
graph TD; A[Bean Definition] --> B[Bean Instantiation]; B --> C[Populate Properties / Dependency Injection]; C --> D[Aware Callbacks]; D --> E[BeanPostProcessor - Before Initialization]; E --> F["@PostConstruct"]; F --> G[InitializingBean.afterPropertiesSet()]; G --> H[Custom init-method]; H --> I[BeanPostProcessor - After Initialization]; I --> J[Bean Ready / In Use]; J --> K["@PreDestroy"]; K --> L[DisposableBean.destroy()]; L --> M[Custom destroy-method];
```

1. Bean Definition

Spring first reads the bean metadata:

@Component

```
public class PaymentService {
```

```
</> Java
```

or:

```
@Bean
public PaymentService paymentService() {
    return new PaymentService();
}
```

The metadata becomes a **BeanDefinition** containing information such as:

- Bean class
- Scope
- Dependencies
- Lazy/eager configuration
- Initialization method
- Destruction method
- Autowiring information

2. Bean Instantiation

Spring creates the bean object.

For example:

```
</> Java
```

For constructor injection:

```
</> Java

@Component
public class PaymentService {

    private final PaymentRepository repository;

    public PaymentService(PaymentRepository repository) {
        this.repository = repository;
    }
}
```

Spring resolves `PaymentRepository` and uses it while constructing `PaymentService`.

3. Dependency Injection / Property Population

[✦ Upgrade](#)

Spring populates dependencies into the bean.

For example:

```
</> Java

@Autowired
private PaymentRepository repository;
```



The dependency is injected during the **populate bean** phase.

For setter injection:

```
</> Java

@Autowired
public void setRepository(PaymentRepository repository) {
    this.repository = repository;
}
```



4. Aware Callbacks

If the bean implements Spring's `Aware` interfaces, Spring provides framework-level information.

Examples:

```
</> Java

BeanNameAware
BeanFactoryAware
ApplicationContextAware
```



Example:

```
</> Java

@Component
public class MyBean implements BeanNameAware {

    @Override
```



```
public void setBeanName(String name) {
    System.out.println(name);
}
}
```

This is mainly useful when a bean needs access to Spring infrastructure.

5. BeanPostProcessor — Before Initialization

Spring invokes:

```
</> Java
postProcessBeforeInitialization()
```

This happens before initialization callbacks such as `@PostConstruct` .

```
</> Java
@Component
public class MyProcessor implements BeanPostProcessor {

    @Override
    public Object postProcessBeforeInitialization(
        Object bean, String beanName) {

        return bean;
    }
}
```

This is an important extension point for modifying/intercepting beans.

6. Initialization

Spring then executes initialization callbacks.

The commonly encountered order is:

```
@PostConstruct
    ↓
InitializingBean.afterPropertiesSet()
    ↓
custom init-method
```

Example:

```
</> Java

@PostConstruct
public void init() {
    // initialization logic
}
```

Or:

```
</> Java

@Override
public void afterPropertiesSet() {
    // initialization logic
}
```

Or:

```
</> Java

@Bean(initMethod = "initialize")
public PaymentService paymentService() {
    return new PaymentService();
}
```

Recommendation: Prefer `@PostConstruct` for normal application-level initialization rather than coupling application code to `InitializingBean`.

7. BeanPostProcessor — After Initialization

Spring invokes:

```
</> Java

postProcessAfterInitialization()
```

This is particularly important because **Spring may return a proxy instead of the original object.**

For example, features such as:

```
@Transactional
@Async
@Cacheable
AOP
```

can involve proxying.

Conceptually:

```
Original Bean
  ↓
Initialization
  ↓
BeanPostProcessor
  ↓
Possibly Proxy
  ↓
Object stored/exposed by Spring
```



This is why understanding `BeanPostProcessor` is important for senior-level Spring interviews.

8. Bean Ready for Use

After initialization and post-processing are complete, the bean is ready to be used.

```
</> Java
```



```
PaymentService service = applicationContext.getBean(PaymentService.class);
```

For a singleton bean, Spring normally keeps the resulting instance/proxy in the singleton cache.

9. Bean Destruction

When the Spring `ApplicationContext` shuts down, Spring invokes destruction callbacks for applicable beans.

Typical order:

```
@PreDestroy
  ↓
DisposableBean.destroy()
  ↓
custom destroy-method
```



Example:

</> Java



```
@PreDestroy
public void cleanup() {
    // release resources
}
```

Or:

</> Java



```
@Override
public void destroy() {
    // cleanup
}
```

Important Interview Point: Complete Lifecycle

You should remember the lifecycle at this level:

1. BeanDefinition created/registered
- ↓
2. Bean instantiated
- ↓
3. Dependencies populated
- ↓
4. Aware callbacks
- ↓
5. BeanPostProcessor.beforeInitialization()
- ↓
6. @PostConstruct
- ↓
7. afterPropertiesSet()
- ↓
8. custom init-method
- ↓
9. BeanPostProcessor.afterInitialization()
- ↓
10. Bean ready
- ↓
11. @PreDestroy
- ↓
12. destroy()
- ↓
13. custom destroy-method



What matters most for a 7–8 year developer

Focus especially on these:

- **BeanDefinition vs Bean instance**
- **Instantiation vs dependency injection**
- **BeanPostProcessor**
- `@PostConstruct`
- `InitializingBean`
- **Custom** `init-method`
- `@PreDestroy`
- `DisposableBean`
- **Bean destruction**
- **Singleton vs prototype lifecycle**
- **Early singleton exposure / circular dependencies**
- **How AOP proxies fit into the lifecycle**
- **Role of** `AbstractAutowireCapableBeanFactory` **and** `initializeBean()`
- **Three-level singleton cache in circular dependency scenarios**

One key distinction: **bean creation and bean initialization are not the same thing.**

Instantiation creates the Java object; dependency population supplies its dependencies; initialization callbacks then prepare it for use.



Before Initialization

Before initialization is the `BeanPostProcessor` phase that runs **after dependency injection** and **Aware callbacks**, but **before the bean's initialization callbacks**.

Spring calls:

`</> Java`

```
postProcessBeforeInitialization(bean, beanName)
```

Lifecycle Position

```
Instantiation
  ↓
Populate Properties
  ↓
Aware callbacks
  ↓
BeanPostProcessor.beforeInitialization()
  ↓
@PostConstruct
  ↓
afterPropertiesSet()
  ↓
Custom init-method
  ↓
BeanPostProcessor.afterInitialization()
  ↓
Bean Ready
```

Example

`</> Java`

```
@Component
public class MyProcessor implements BeanPostProcessor {
```

```
@Override
public Object postProcessBeforeInitialization(
    Object bean, String beanName) {

    System.out.println("Before initialization: " + beanName);

    return bean;
}
```

This processor gets a chance to **inspect or modify the bean before its initialization callbacks execute**.

Important

`postProcessBeforeInitialization()` is commonly used by Spring infrastructure for lifecycle processing.

For example, `@PostConstruct` processing is integrated into the `BeanPostProcessor` mechanism.

Key Point

Before initialization does not mean before dependency injection.

The bean has already been instantiated and its dependencies populated.

Object created
↓
Dependencies injected ✓
↓
BeforeInitialization
↓
@PostConstruct



Also, **returning the same bean is normal**. A `BeanPostProcessor` can return a different object, but proxy creation is typically associated with **post-initialization processing**.

Interview Definition

`postProcessBeforeInitialization()` is a `BeanPostProcessor` **callback that allows Spring to process a bean after dependency injection but before its initialization callbacks such as `@PostConstruct`**.

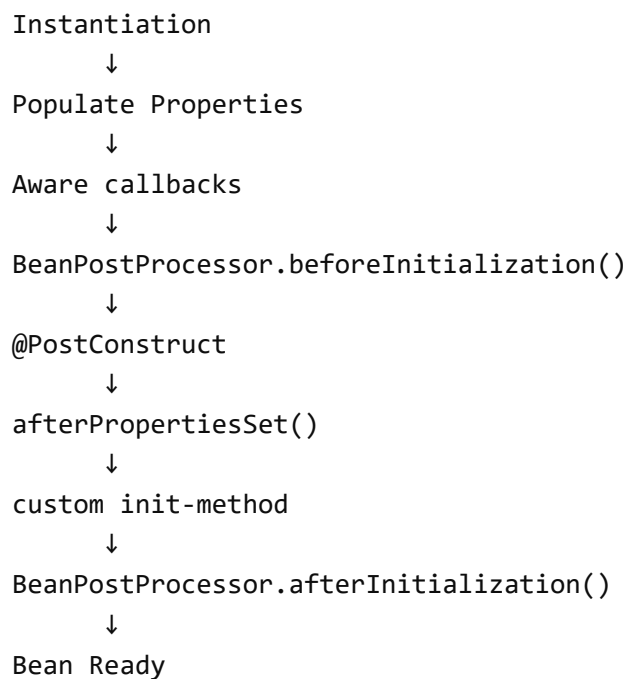


- BeanPostProcessor

BeanPostProcessor

BeanPostProcessor is a **Spring extension point** that allows you to process or modify beans before and/or after initialization.

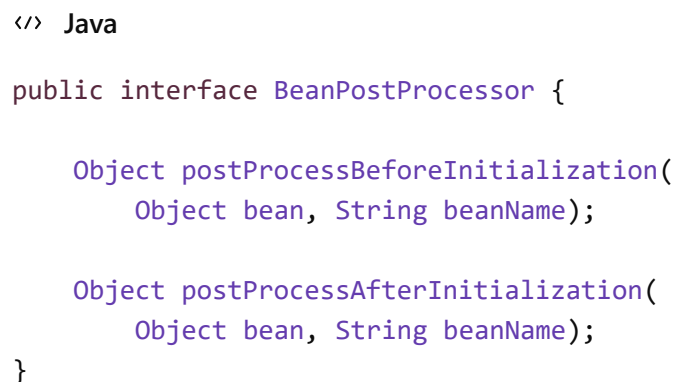
Lifecycle Position



```
graph TD; A[Instantiation] --> B[Populate Properties]; B --> C[Aware callbacks]; C --> D[BeanPostProcessor.beforeInitialization()]; D --> E["@PostConstruct"]; E --> F[afterPropertiesSet()]; F --> G[custom init-method]; G --> H[BeanPostProcessor.afterInitialization()]; H --> I[Bean Ready];
```

The diagram illustrates the Spring bean lifecycle flow. It starts with 'Instantiation', followed by 'Populate Properties', 'Aware callbacks', and then 'BeanPostProcessor.beforeInitialization()'. This is followed by '@PostConstruct', 'afterPropertiesSet()', 'custom init-method', 'BeanPostProcessor.afterInitialization()', and finally 'Bean Ready'. Arrows indicate the sequential flow between these steps.

Main Methods



```
</> Java

public interface BeanPostProcessor {

    Object postProcessBeforeInitialization(
        Object bean, String beanName);

    Object postProcessAfterInitialization(
        Object bean, String beanName);
}
```

The code snippet shows the Java interface for BeanPostProcessor. It includes two methods: postProcessBeforeInitialization and postProcessAfterInitialization, both of which take an Object bean and a String beanName as parameters and return an Object.

Both methods receive the bean and can return:

- the same bean, or
- a **different object** such as a proxy.

Example

```
</> Java

@Component
public class MyBeanPostProcessor implements BeanPostProcessor {

    @Override
    public Object postProcessBeforeInitialization(
        Object bean, String beanName) {

        // process bean
        return bean;
    }

    @Override
    public Object postProcessAfterInitialization(
        Object bean, String beanName) {

        // process bean
        return bean;
    }
}
```

Why is it important?

Many Spring features are built around `BeanPostProcessor`, including processing related to:

```
@Autowired
@PostConstruct
AOP
@Transactional
@Async
```

A key reason is **proxy creation**.

Conceptually:

```
Original Bean
  ↓
BeanPostProcessor
  ↓
Proxy created
  ↓
Proxy exposed as Spring bean
```

For example, with:

</> Java



```
@Transactional
public void transfer() {
}
```

Spring can create a proxy around the bean so that transaction behavior happens when the method is invoked.

Most Important Interview Point

BeanPostProcessor operates on individual bean instances and provides hooks before and after initialization. It is one of the primary extension points Spring uses to customize beans and create proxies.

Also remember:

BeanFactoryPostProcessor $\neq$ **BeanPostProcessor**

- **BeanFactoryPostProcessor** → modifies **BeanDefinitions/metadata**
- **BeanPostProcessor** → processes **actual bean instances**



- **InstantiationAwareBeanPostProcessor**
- Initialization callbacks
 - **@PostConstruct**
 - **InitializingBean**
 - **init-method**
- After initialization
- Destruction callbacks
 - **@PreDestroy**
 - **DisposableBean**
 - **destroy-method**

Show more ▾

For a 7–8 year experienced Spring developer, these are the **important lifecycle details** to know.

1. InstantiationAwareBeanPostProcessor

`InstantiationAwareBeanPostProcessor` is an advanced extension of `BeanPostProcessor` that allows Spring to **intervene during bean instantiation and dependency population**.

It provides callbacks such as:

</> Java



```
postProcessBeforeInstantiation()  
postProcessAfterInstantiation()  
postProcessProperties()
```

Why important?

It can influence:

- Bean instantiation
- Dependency/property population
- Field injection
- Whether the normal instantiation process should proceed

A major Spring implementation is:

`AutowiredAnnotationBeanPostProcessor`



which handles annotations such as:

</> Java



```
@Autowired  
@Value
```

Key distinction

`BeanPostProcessor`



→ processes bean before/after initialization

`InstantiationAwareBeanPostProcessor`

→ can intervene earlier during instantiation
and dependency population

2. Initialization Callbacks

Initialization callbacks execute **after dependency injection** and before the bean is considered fully initialized.

Common mechanisms:

```
@PostConstruct
    ↓
InitializingBean.afterPropertiesSet()
    ↓
custom init-method
```



These are used for initialization logic such as:

- Validating configuration
- Initializing internal state
- Preparing resources

3. @PostConstruct

`@PostConstruct` is the preferred application-level initialization callback.

```
</> Java

@PostConstruct
public void init() {
    // initialization logic
}
```



It executes after dependencies have been populated.

Important

It is not Spring-specific; it is a Jakarta lifecycle annotation:

```
</> Java

jakarta.annotation.PostConstruct
```



4. InitializingBean

A bean can implement:

```
</> Java
```



InitializingBean

and override:

```
</> Java

@Override
public void afterPropertiesSet() {
    // initialization logic
}
```

Why less preferred?

It couples your application class directly to Spring.

Therefore:

```
@PostConstruct      → preferred
InitializingBean     → useful when Spring-specific lifecycle coupling is a
```

5. init-method

You can configure a custom initialization method.

```
</> Java

@Bean(initMethod = "init")
public PaymentService paymentService() {
    return new PaymentService();
}
```

Bean:

```
</> Java

public void init() {
    // initialization
}
```

This is useful when you **cannot modify the class** or don't want the class to implement Spring lifecycle interfaces.

6. Initialization Order

For a typical bean:

```

Instantiation
  ↓
Dependency Injection
  ↓
Aware callbacks
  ↓
BeanPostProcessor.beforeInitialization()
  ↓
@PostConstruct
  ↓
InitializingBean.afterPropertiesSet()
  ↓
init-method
  ↓
BeanPostProcessor.afterInitialization()
  ↓
Bean ready
  
```

The important distinction is:

BeanPostProcessor surrounds the initialization callbacks.

7. After Initialization

After initialization callbacks complete, Spring invokes:

```
</> Java
```

```
postProcessAfterInitialization()
```

from BeanPostProcessor .

This is an important extension point because Spring can **replace the original bean with another object**, commonly a proxy.

Conceptually:

```

Original Bean
  ↓
Initialization
  ↓
postProcessAfterInitialization()
  ↓
Proxy / processed bean
  ↓
  
```

Bean exposed to application

This is important for Spring features such as:

@Transactional
@Async
AOP



Key point

Don't assume:

```
</> Java  
  
getBean()
```



always returns the exact object created during instantiation.

It may return a **Spring-generated proxy**.

8. Destruction Callbacks

When the `ApplicationContext` shuts down, Spring invokes destruction callbacks for applicable beans.

Typical order:

```
@PreDestroy  
↓  
DisposableBean.destroy()  
↓  
destroy-method
```



These are used for cleanup such as:

- Closing resources
- Stopping background tasks
- Releasing connections/resources

9. @PreDestroy

Preferred application-level destruction callback:

</> Java



```
@PreDestroy
public void cleanup() {
    // cleanup
}
```

Example:

</> Java



```
@Component
public class ConnectionManager {

    @PreDestroy
    public void close() {
        System.out.println("Closing resources");
    }
}
```

It executes when the Spring container destroys the bean.

10. DisposableBean

A bean can implement:

</> Java



```
DisposableBean
```

and override:

</> Java



```
@Override
public void destroy() {
    // cleanup
}
```

Like `InitializingBean`, this creates Spring-specific coupling.

So normally:

`@PreDestroy` → preferred

`DisposableBean` → when Spring-specific lifecycle coupling is acceptable



11. destroy-method

You can specify a custom destruction method:

```
</> Java

@Bean(destroyMethod = "close")
public ConnectionManager connectionManager() {
    return new ConnectionManager();
}
```

Spring calls:

```
</> Java

connectionManager.close();
```

when the context shuts down.

Useful when the class already provides a cleanup method such as:

```
</> Java

close()
shutdown()
stop()
```

12. Prototype Destruction Limitation

This is very important for interviews.

For a **singleton** bean:

```
Spring creates it
  ↓
Spring manages it
  ↓
Spring destroys it
```

For a **prototype** bean:

```
Spring creates it
  ↓
Spring gives it to caller
```

↓
Caller owns it

Spring **does not** manage the complete destruction lifecycle of prototype beans.

Therefore, Spring does not automatically invoke:

```
@PreDestroy  
DisposableBean.destroy()  
destroy-method
```



for prototype beans when the application context shuts down.

Example

```
</> Java  
  
@Scope("prototype")  
@Component  
public class MyBean {  
  
    @PreDestroy  
    public void cleanup() {  
        System.out.println("Cleanup");  
    }  
}
```



If you repeatedly call:

```
</> Java  
  
context.getBean(MyBean.class);
```



Spring creates multiple instances.

When the context shuts down, Spring does **not** automatically track and destroy all those prototype instances.

Why?

Because Spring's responsibility for a prototype bean effectively ends after:

```
create → configure → initialize → return
```



The application is responsible for cleanup.

Final Lifecycle Picture



```
BeanDefinition
  ↓
Instantiation
  ↓
InstantiationAwareBeanPostProcessor
  ↓
Populate Properties / Dependency Injection
  ↓
Aware Callbacks
  ↓
BeanPostProcessor
beforeInitialization()
  ↓
@PostConstruct
  ↓
InitializingBean.afterPropertiesSet()
  ↓
init-method
  ↓
BeanPostProcessor
afterInitialization()
  ↓
Bean Ready / Possibly Proxy
  ↓
  |   Application runs
  |
  ↓
@PreDestroy
  ↓
DisposableBean.destroy()
  ↓
destroy-method
```

What you should remember for interviews

Concept	Purpose
InstantiationAwareBeanPostProcessor	Intervene during instantiation/property population
@PostConstruct	Initialization callback
InitializingBean	Spring-specific initialization callback
init-method	Custom initialization method
BeanPostProcessor.afterInitializatio n()	Final bean processing; proxy creation can happen here

Concept	Purpose
@PreDestroy	Destruction callback
DisposableBean	Spring-specific destruction callback
destroy-method	Custom cleanup method
Prototype	Spring does not automatically perform destruction callbacks

Most important distinction:

BeanPostProcessor

↓

Actual bean lifecycle processing

BeanFactoryPostProcessor

↓

BeanDefinition / metadata processing

And:

Singleton → Spring manages creation + destruction

Prototype → Spring manages creation, but NOT destruction



ChatGPT can make mistakes. Check important info.